

Intelligenza Artificiale

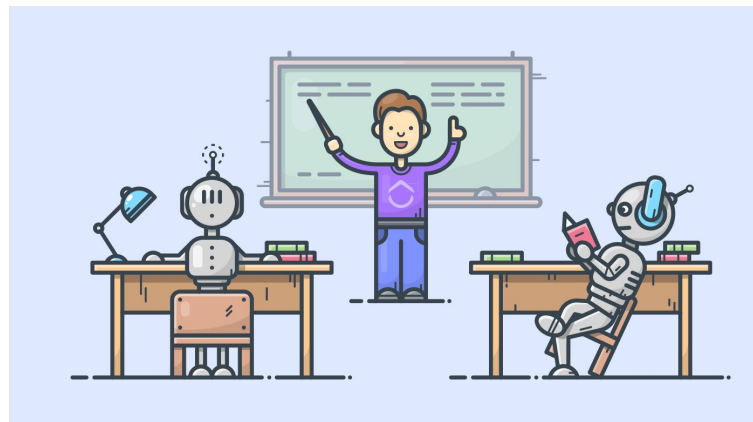
“Deep learning (part 3)”

Nicola Bellotto

A.A. 2025/2026

Lecture outline

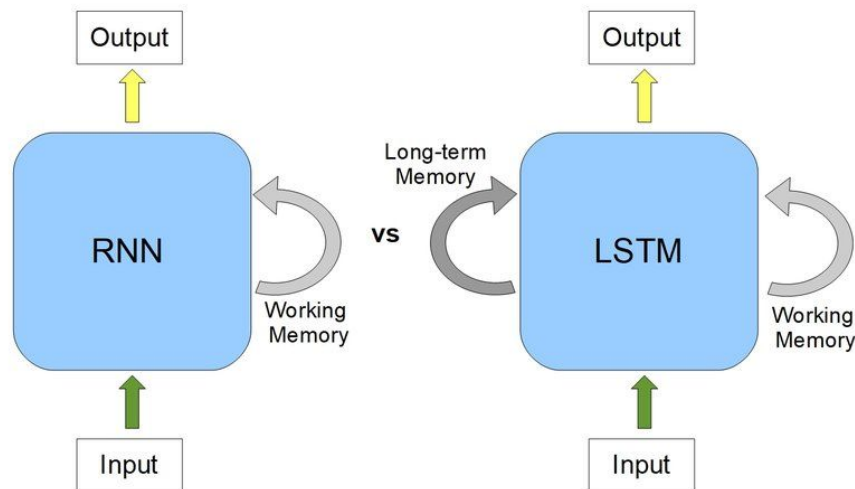
- LSTM
- Unsupervised learning
 - Autoencoders
 - Variational autoencoders
 - GANs



Slides partly based on Russell & Norvig instructors material <http://aima.cs.berkeley.edu/instructors.html>

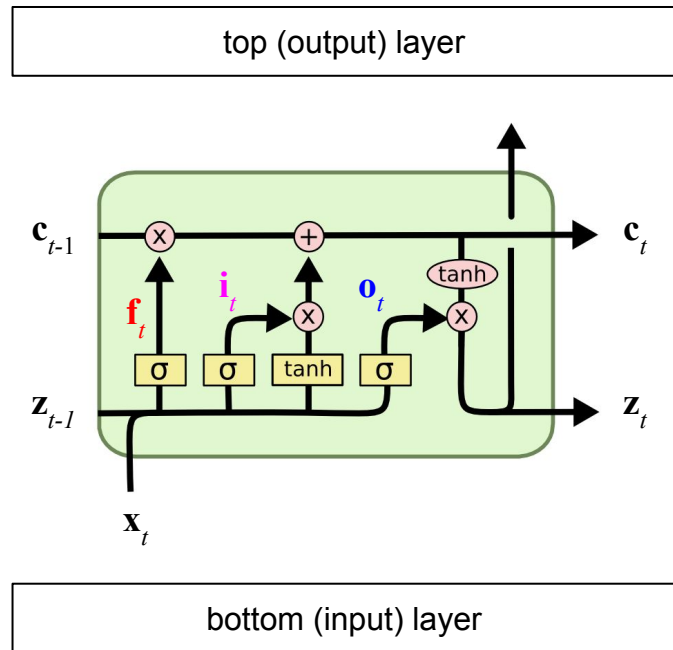
Long short-term memory (LSTM)

- **LSTM** is a type of RNN that augments hidden state $\mathbf{z}_t$ and preserves information over time
- Has a **long-term memory** component, **memory cell** ($\mathbf{c}$) copied from time step to time step, instead of being multiplied
 - this avoid the gradient problems



Long short-term memory (LSTM)

- LSTMs have **gating units**:
 - vectors to control the flow of information
 - used for element-wise multiplication of the corresponding information vector
- Types of gating units:
 - **forget gate** (**f**) determines if each element of the memory cell is remembered
 - **input gate** (**i**) determines if each element of the memory cell is updated additively
 - **output gate** (**o**) determines if each element of the memory cell is transferred to the short-term memory **z**

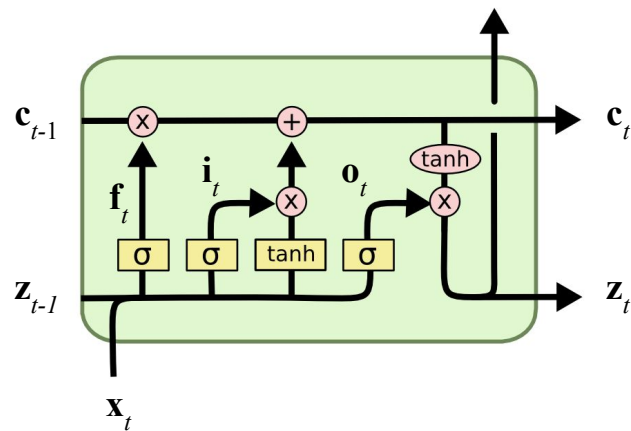


Long short-term memory (LSTM)

$$\begin{aligned} \mathbf{f}_t &= \sigma(\mathbf{W}_{x,f}\mathbf{x}_t + \mathbf{W}_{z,f}\mathbf{z}_{t-1}) \\ \mathbf{i}_t &= \sigma(\mathbf{W}_{x,i}\mathbf{x}_t + \mathbf{W}_{z,i}\mathbf{z}_{t-1}) \\ \mathbf{o}_t &= \sigma(\mathbf{W}_{x,o}\mathbf{x}_t + \mathbf{W}_{z,o}\mathbf{z}_{t-1}) \\ \mathbf{c}_t &= \mathbf{c}_{t-1} \odot \mathbf{f}_t + \mathbf{i}_t \odot \tanh(\mathbf{W}_{x,c}\mathbf{x}_t + \mathbf{W}_{z,c}\mathbf{z}_{t-1}) \\ \mathbf{z}_t &= \tanh(\mathbf{c}_t) \odot \mathbf{o}_t \end{aligned}$$

Diagram illustrating the LSTM cell structure and its components:

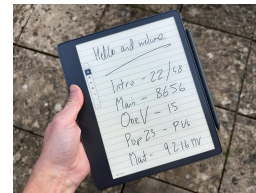
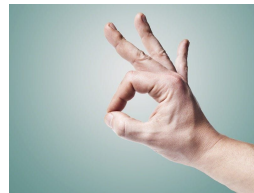
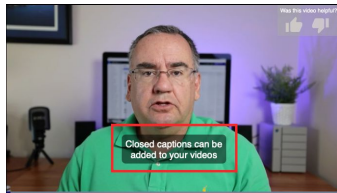
- forget**: Points to the $\mathbf{c}_{t-1} \odot \mathbf{f}_t$ term in the cell state update equation.
- input**: Points to the $\mathbf{i}_t \odot \tanh(\mathbf{W}_{x,c}\mathbf{x}_t + \mathbf{W}_{z,c}\mathbf{z}_{t-1})$ term in the cell state update equation.
- output**: Points to the $\tanh(\mathbf{c}_t) \odot \mathbf{o}_t$ term in the hidden state update equation.



- Gates in LSTM differ from Boolean functions, they have range $[0,1]$ and sigmoid outputs

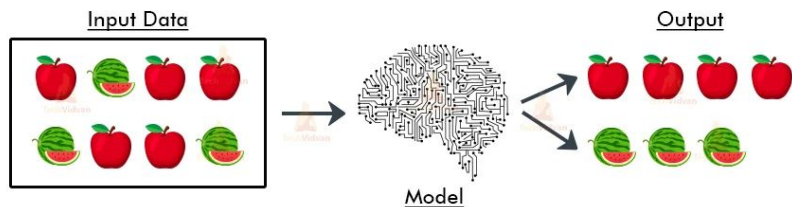
LSTM applications

- Applications of LSTM:
 - **Natural language processing (NLP):** language modeling, sentiment analysis, machine translation
 - **Time series analysis:** stock prediction, weather forecasting, demand forecasting
 - **Speech recognition:** converting spoken language to text
 - **Image and video captioning:** generating descriptions for visual content
 - **Gesture recognition:** interpreting and classifying human gestures
 - **Anomaly detection:** fraud detection, network intrusion detection
 - **Handwriting recognition:** converting handwritten text to digital



Unsupervised learning

- **Supervised learning**
 - Each input x in training set has an associated output y
 - Goal is to learn the input-output mapping
- **Unsupervised learning** only takes training sets of unlabeled examples x
 - Just input x without corresponding output y



- Unsupervised learning avoids need of annotating large datasets for training, sometime very hard (e.g. medical images)
- Also no need to partition the world into often arbitrary categories
 - E.g. labeling “*drinking*” or “*sipping*” actions on a video: when person picks up the glass? or when glass touches the mouth? or when the liquid pours out? ...
- Unsupervised learning forces the model to “explain” high-dimensional inputs, not just low-dimensional outputs

Goal of unsupervised learning

- Two types of use
 1. to learn **new representations** – e.g. key features of an image that helps a classification task
 2. to learn **generative models** – e.g. probability distribution $\sim P(\mathbf{x})$ used to generate new examples
- E.g. a generative model that learned different (hidden) *features* $\mathbf{z}$ to represent faces can be used to perform arithmetic operations in the $\mathbf{z}$ space
 - start with features of “*man with glasses*”
 - subtract coordinates for “*man*”
 - add coordinates for “*woman*”



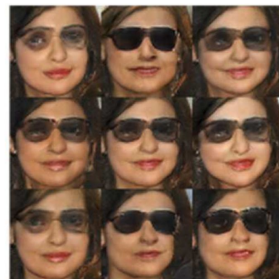
–



+



=



Autoencoders

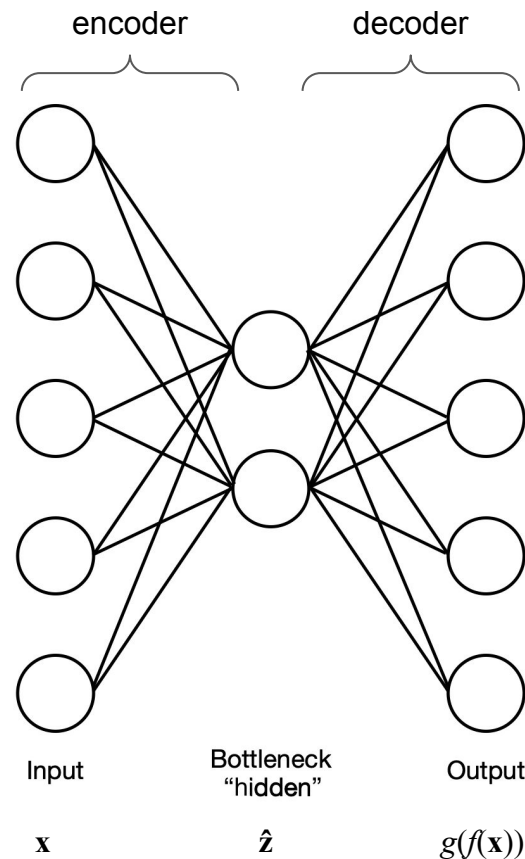
- **Autoencoder (AE)** – unsupervised model containing
 - an **encoder** f that maps from input $\mathbf{x}$ to an internal *bottleneck* representation $\hat{\mathbf{z}}$, also called **embedding**
 - a **decoder** g that maps from $\hat{\mathbf{z}}$ back to observed data $\mathbf{x}$

$$\mathbf{x} \approx g(f(\mathbf{x}))$$

- **Linear autoencoder** – simple AE where both f and g are linear (use unit activation functions) with a shared matrix $\mathbf{W}$

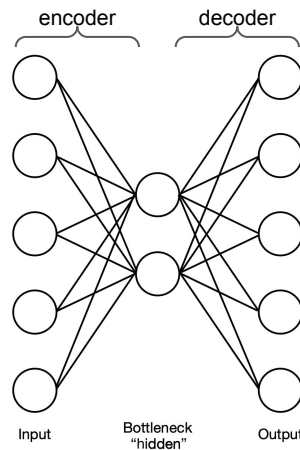
$$\hat{\mathbf{z}} = f(\mathbf{x}) = \mathbf{W}\mathbf{x}$$

$$\mathbf{x} = g(\hat{\mathbf{z}}) = \mathbf{W}^\top \hat{\mathbf{z}}$$



AE applications

- The **encoder** alone can be used as a component of a different neural network (e.g. for image classification)
 - E.g. for **data compression** and **dimensionality reduction**
 - extract features for other tasks
 - increase computational speed and efficiency
- The **decoder** part is useful to generate data, e.g. for
 - **image/audio denoising**: remove noise/errors that do not match the latent space representation learned by the network
 - **image reconstruction**: fill-in missing elements of or colorize images
 - **anomaly detection**: compare differences between input and output



AE example (from K. P. Murphy “Probabilistic Machine Learning” <https://probml.github.io/pml-book/>)

- AE trained on the [Fashion MNIST dataset](#)
 - Fully-connected feedforward architecture with 2 layers and a bottleneck of size 30
 - Some test images and their reconstruction

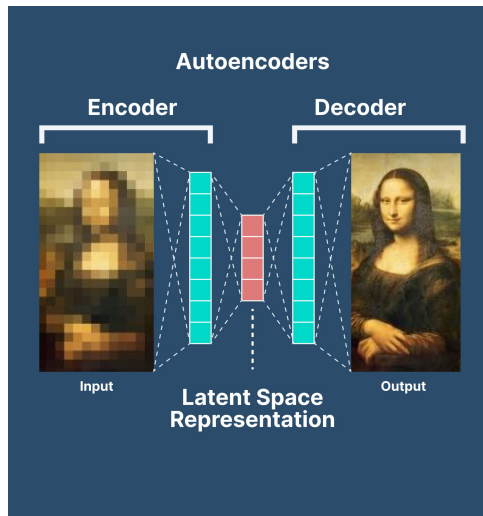


- First 2-out-of-30 *latent* dimensions of the AE, color coded by (unused) class labels
- AE does a good job in separating the classes in a fully unsupervised way

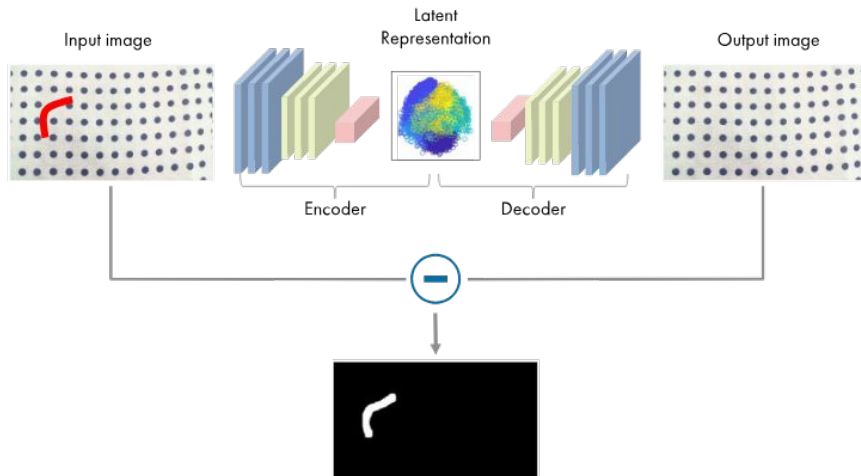


AE examples

image denoising

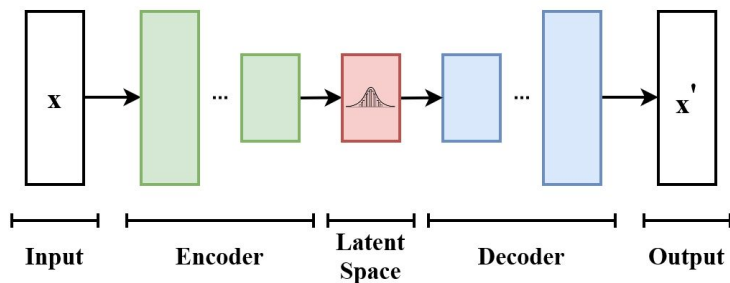


anomaly detection



Variational autoencoder (VAE)

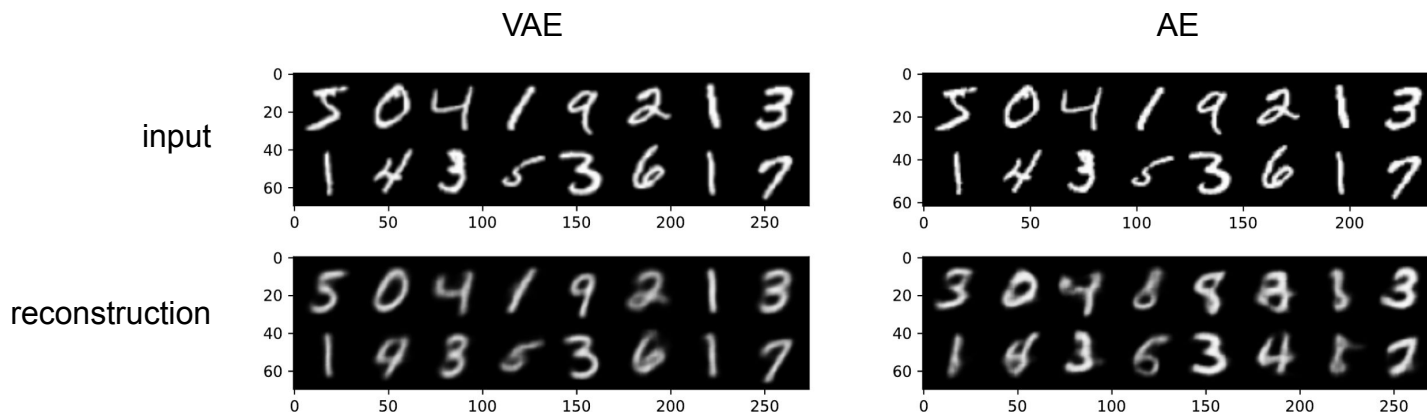
- **Variational autoencoder (VAE)**: more complex network, can capture more complex types of generative models
 - Essentially a probabilistic version of an AE, which can generate **new** examples



- ➡ Whereas the AE's inputs are encoded in (discrete) points $\mathbf{z}$ of the latent space, the VAE encodes (continuous) probability distributions $P(\mathbf{z} | \mathbf{x})$ (e.g. Gaussians)

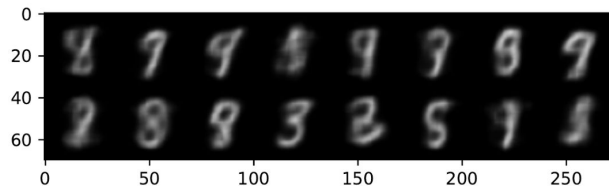
VAEs vs AEs – reconstruction

- Similar or better reconstruction abilities of VAE compared to AE
 - Example: reconstruction of [MNIST digits](#) using a 20 dimensional latent space

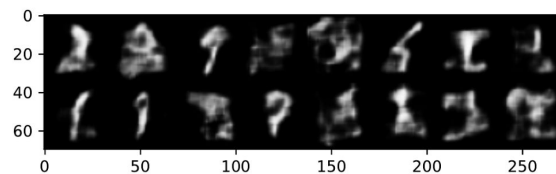


VAEs vs AEs – data generation

- Primary advantage of VAE is that it can generate new data from random noise
 - (a) **VAE's** decoder is trained to convert random points in the embedding space (generated by perturbing the input encodings) to sensible outputs
 - (b) **AE's** (deterministic) decoder only ever gets as inputs the *exact* encodings of the training set, so it cannot understand random inputs outside what it was trained on $\Rightarrow$ standard AE cannot create (completely) new samples



(a)

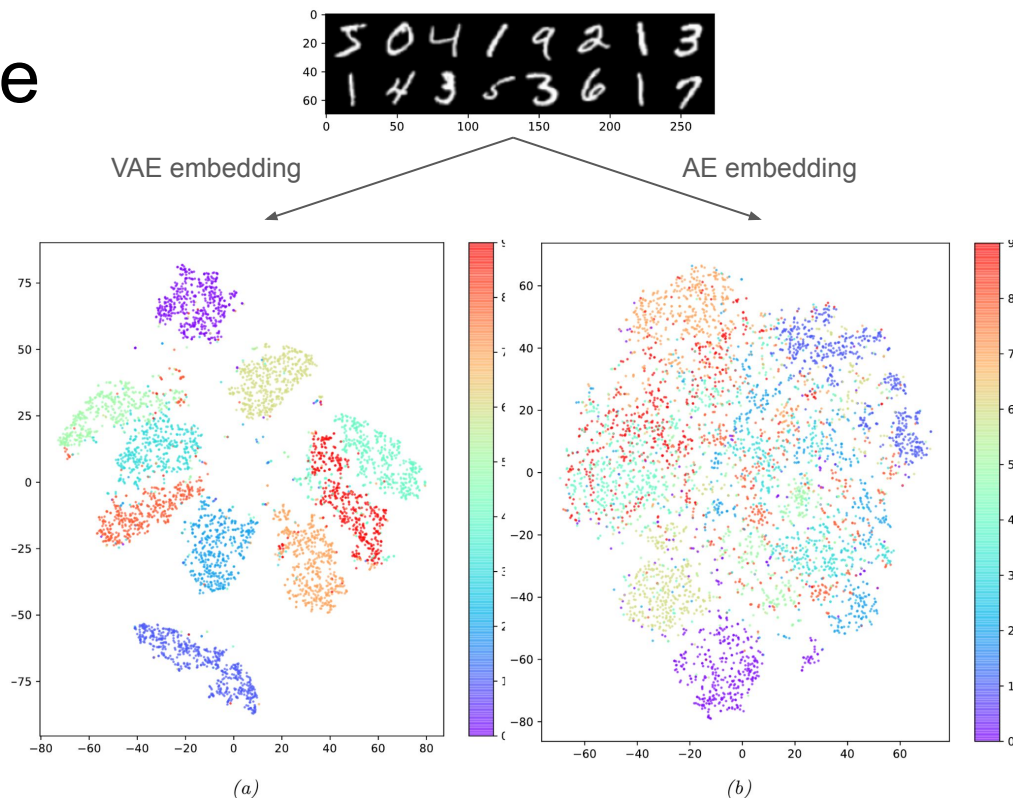


(b)

VAEs vs AEs – example

E.g. image classification

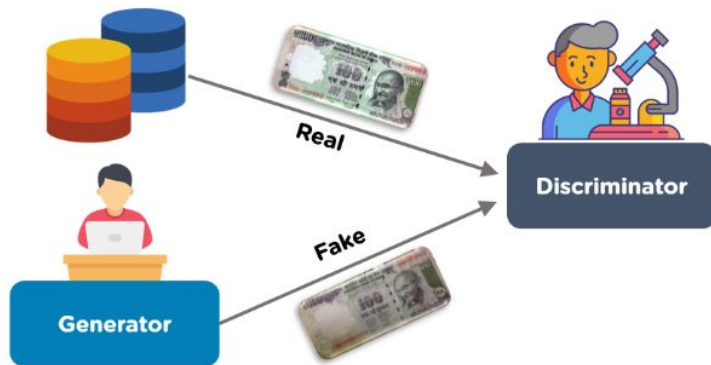
- VAE better at sampling because it embeds images in (latent) Gaussians, whereas AE embeds images in points
- a.* With VAE's latent distributions, images from the same class are mapped to clusters of same classes...
- b.* ... instead with AE, images from different classes can end up all mixed and next to each other



E.g. multi-dimensional latent space with VAE (a) and AE (b)
(from K. P. Murphy “Probabilistic Machine Learning” – <https://probml.github.io/pml-book/>)

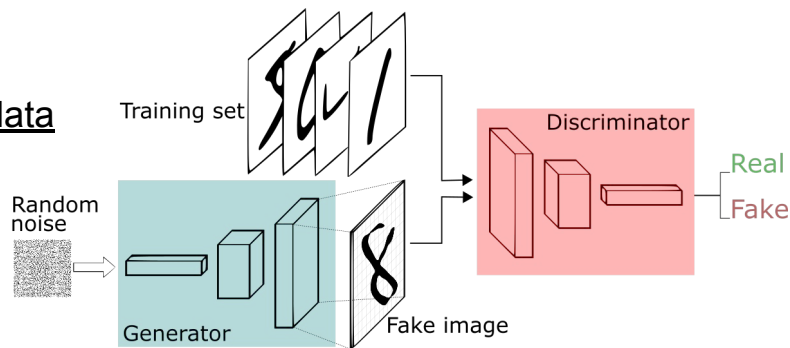
Generative adversarial networks (GANs)

- Pair of networks combined to form a generative system
 - the **generator** maps **fake** values from z to x
 - the **discriminator** is a classifier trained to classify inputs x as **real** or **fake** (generated)



GAN properties

- GANs are *implicit* models:
 - they do not explicitly model probabilities like VAEs, nor provide a means for finding the latent variable corresponding to a given sample like AEs
 - samples can be generated, but their probabilities are not readily available
- Generator and discriminator are trained simultaneously
 - **Generator** learns to *fool* the discriminator
 - **Discriminator** learns to *separate* real from fake data



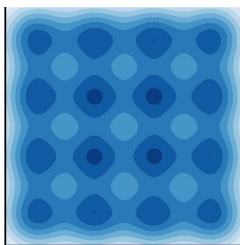
GANs application examples

- GANs work particularly well for image generation tasks
 - For example, can create photorealistic, high-resolution images of people who have never existed (e.g. <https://thispersondoesnotexist.com/>)



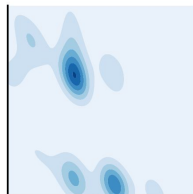
Problems with GANs

- During training, GANs can suffer from **mode collapse**
 - generator converges to distribution which does not cover all the modes (peaks) of the data distribution (underfitting)
- Another problem is **mode hopping**, where generator “hops” between a few different modes of the data distribution
 - generator becomes good in one mode, where discriminator is bad, and simply hops when necessary...
- **Solutions:** larger batch sizes; bigger discriminators; regularization (reduce complexity to improve generalisation); more optimizations



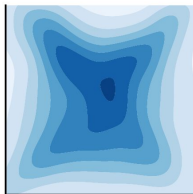
(a) Data

Disc Loss: 0.892
Gen Loss: 1.852



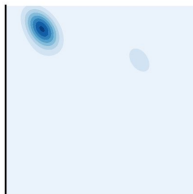
(d) 8000 iterations

Disc Loss: 1.252
Gen Loss: 0.753



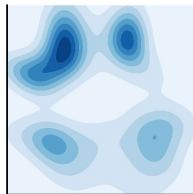
(b) 2000 iterations

Disc Loss: 0.457
Gen Loss: 3.258



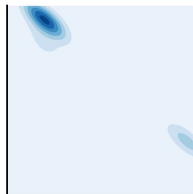
(e) 12000 iterations

Disc Loss: 1.136
Gen Loss: 0.740



(c) 4000 iterations

Disc Loss: 0.305
Gen Loss: 3.016



(f) 14000 iterations

Conclusions

- Suggested reading
 - Chapter 22, sec. 22.6–22.7 of Russell & Norvig
- Next lecture
Deep learning (part 4)
- Questions?

